

Review

A question from last week's lab: Implement a function `print_list` which prints out all the elements of a nested list.

For example, if we have

```
lst = [["hello"], "darkness", ["my", "old", ["friend"]],
```

it should print:

```
hello darkness my old friend

# to print_list lst, we could
# - either go over lst, print each x in lst
# - or print lst[0], and print the rest

def print_list(lst):
    if type(lst) == str:
        print(lst, end = " ")
    else:
        for x in lst:
            print_list(x)

def print_list2(lst):
    if type(lst) == str:
        print(lst, end = " ")
    else:
        if lst != []:
            print_list2(lst[0])
            print_list2(lst[1:])

print_list2([["hello"], "darkness", ["my", "old", ["friend"]]])

hello darkness my old friend
```

A question from last week's quiz: Write a function called `sum_of_digits(n)` that calculates the sum of the digits of a number `n`.

```
def sum_of_digits(n):
    # n=41342316547
    # sum of digits of n = last digit + sum of digits of (all but the
    # last digit of n)
    if (n < 10):
        return n
    else:
        return n % 10 + sum_of_digits(n // 10)
```

```
sum_of_digits(34134)
```

```
15
```

Introduction to Algorithms

For additional reading and resources, you can refer to <https://jeffe.cs.illinois.edu/teaching/algorithms/book/00-intro.pdf> and <https://jeffe.cs.illinois.edu/teaching/algorithms/book/01-recursion.pdf>

What is an algorithm?

- Sequence of precise instructions, usually intended to accomplish a specific purpose
- Not language specific i.e. you can use any programming language to implement an algorithm

Algorithm examples?

- In real life, this might be represented by the path you take to get from your dorm to the lecture hall. The set of steps you take is an algorithm.
- To tie it back to what you've already learned, you have previously written functions to do a particular task. Inside these functions, you have instructions for doing the task. The logic you implement in the functions is an algorithm

Let's look at an example below:

```
# In week 2, you did Fibonacci sequence in lab:
# We learned Fibonacci sequence is represented by  $F(n) = F(n - 1) + F(n - 2)$ 
# We can write this as the function below:

def fib(n):    # time complexity:  $O(F(n))$ 
    if (n < 0):
        return -1
    if (n <= 1):
        return n
    return fib(n-1) + fib(n-2)
```

If we run this function, we see we can put in different inputs, but get different results:

```
print(fib(4)) # First, we try with 4 as the input
3
print(fib(2)) # Then, we try with 2 as the input
1
```

These are the **SAME** function, but produce **DIFFERENT** outputs. However, in both cases, we use the **SAME** algorithm to find the right Fibonacci number

`fib(n)` implements an algorithm that computes the n -th Fibonacci number.

Here is another function that computes the Fibonacci numbers

```
def fib2(n):  
    fib_num = [0] * (n+1)  
    fib_num[0] = 0  
    fib_num[1] = 1  
    for i in range(2, n+1):  
        fib_num[i] = fib_num[i-1] + fib_num[i-2]  
    return fib_num[n]
```

```
print(fib2(4))
```

```
3
```

```
print(fib2(2))
```

```
1
```

`fib2(n)` also implements an algorithm that computes the n -th Fibonacci number.

It turns out that `fib(n)` and `fib2(n)` implement **DIFFERENT** algorithms for the **SAME** task.

Why do we care about algorithms?

Let's go back to the real life example of finding a path from the dorm to the lecture hall. Let's pretend we can represent it with the image below:

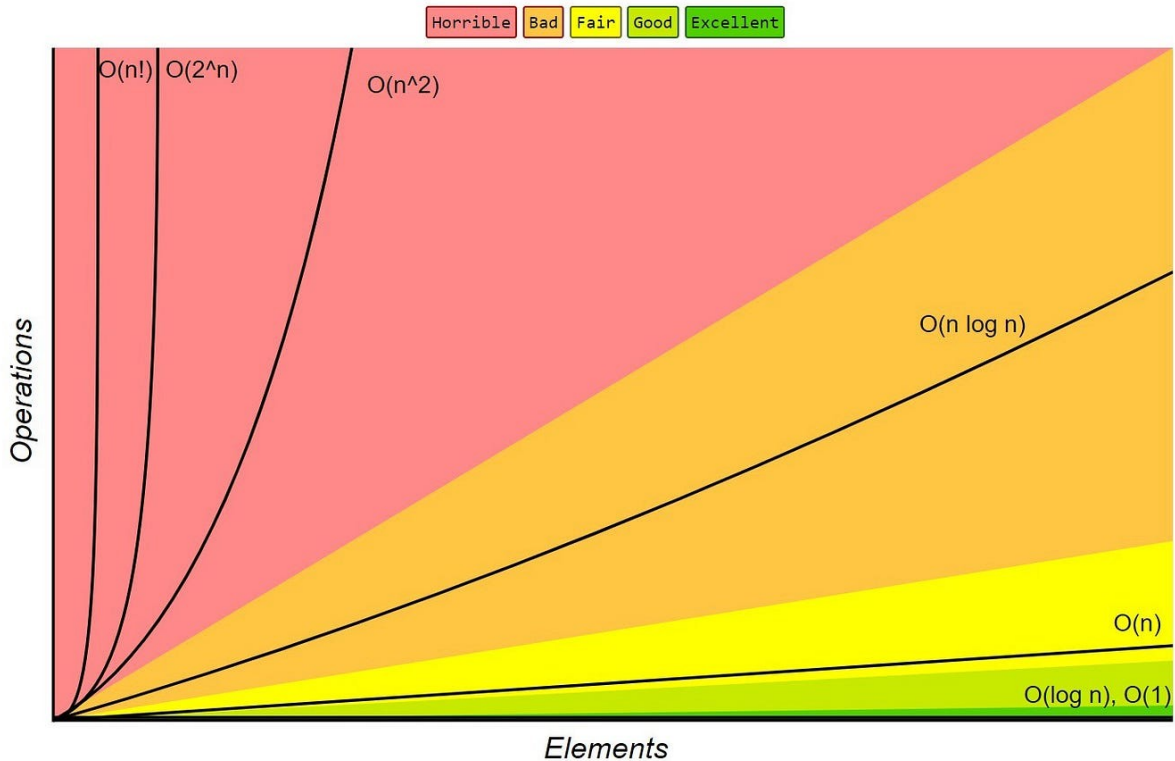
screenshot%202024-07-25%20211131.png

In the example, we have 3 paths that a student can take to get from the dorm to the lecture hall. Each of these is an algorithm. However, how do we know which of these is the best path to take if I want to get to the lecture hall the fastest?

Defining "best" algorithm

Last week, you started talking about time complexity. Time complexity is a way to measure how fast an algorithm runs in a more consistent way. Since computer hardware can affect how fast a piece of code runs, instead of defining the speed of an algorithm through time, we define it in terms of the number of operations. Below is an example of different time complexities:

Big-O Complexity Chart



This should give you an understanding of what different time complexities mean. For example, we see as the number of elements increases, $O(N^2)$ starts requiring a lot more operations than something like $O(N)$.

While there are many ways to describe time complexity (average, best case, worst case), we will focus on worst case time complexity for this class which is represented with Big-O notion (i.e. $O(N)$). The "best" algorithm will then be the algorithm that has the smallest worst case time complexity.

```
def fun(arr):    #  $O(n^2)$ 
    n = len(arr) #  $O(1)$ 
    result = [0] * n #  $O(n)$ 
    for i in range(n):
        result[i] = max(arr[:i+1]) #  $O(n) * O(n), O(2n) = O(n)$ 
    return result
```

What is the time complexity of the function above? Let's break it down into parts:

1. Consider `result[i] = max(arr[:i+1])`. In the worse case, what would be time complexity of this line? *Hint: we are trying to find the MAX element of an array and when we slice, we are just copying the array*
2. Consider `for i in range(n):`. What is the time complexity of this line?

3. Consider `n = len(arr)`. What is the time complexity of this line?